

DB LAB FINAL VIVA PREP - MAY 6, 2026

Multi-Tenant CRM System Questions and Simple Answers

Use this as your quick speaking sheet for the demo. The answers are written for your actual project: TenantCRM / MultiTenant_CRM using HTML, CSS, JavaScript, Node.js, Express, and SQL Server.

Goal: explain design, show working features, and answer database concepts confidently.

3**MAIN ROLES**

Super Admin,
Tenant Admin, Sales
Agent.

18+**FEATURES**

More than the
required 15
functional features.

SQL Server**DATABASE**

Database name
used in code:
MultiTenant_CRM.

CRUD**COVERAGE**

Create, read,
update, delete
across major
modules.

One-minute introduction to say first

My project is a SaaS-style Multi-Tenant CRM. Different companies can use the same system, but their data is separated by tenant_id. Super Admin manages the whole platform, Tenant Admin manages one company, and Sales Agent manages assigned leads, contacts, activities, opportunities, tickets, and notifications. The frontend is built with HTML, CSS, and JavaScript, the backend is Node.js with Express routes, and the database is SQL Server.

Best 15-minute demo flow

First 7 minutes

- Open landing page and login page.
- Login and show role-based redirection.
- Show dashboard cards, revenue, and pipeline values.
- Show tenants, plans, users, leads, and contacts.

Next 8 minutes

- Add, update, and delete one safe demo record.
- Show opportunities with pipeline stages.
- Show tickets with comments and resolved status.
- Finish with audit logs, notifications, and lead conversion.

TenantCRM Viva Prep - Page 1

Features You Can Claim

Authentication

Role dashboards

Tenant management

Subscription plans

User management

Leads

Contacts

Accounts

Opportunities

Pipeline stages

Activities

Support tickets

Ticket comments

Billing records

Audit logs

Notifications

Custom fields

Email integrations

Lead conversion

Module	Main table or route	Simple explanation for viva
Login	Users, Tenants, /auth/login	Checks username and password, joins tenant name, updates last_login, then redirects by role.
Tenants	Tenants	Stores each company using the CRM. tenant_id separates company data.
Plans	Subscription_Plans	Stores pricing limits such as max users, max contacts, and monthly price.
Users	Users	Stores people who login, their roles, active status, and tenant_id.
Leads	Leads	Stores potential customers before they become contacts or opportunities.
Contacts	Contacts	Stores customer persons linked with accounts and assigned users.
Accounts	Accounts	Stores customer companies with industry, country, revenue, and assigned user.
Opportunities	Opportunities, Pipeline_Stages	Stores possible deals, value, stage, expected close date, and status.
Activities	Activities	Stores calls, meetings, tasks, due dates, and completion status.
Tickets	Support_Tickets, Ticket_Comments	Stores customer support issues and their comments.
Billing	Billing_Records	Stores invoices, payment status, amount, due date, and payment method.
Audit	Audit_Logs	Stores history of important actions so admins can trace changes.
Lead conversion	Lead_Conversions	Links a lead to created contact, account, opportunity, and converted user.

Architecture answer

Frontend pages call Express API routes using fetch. Express receives JSON, validates inputs, uses parameterized SQL Server queries, and returns JSON back to the page. The database stores all permanent data and relationships.

TenantCRM Viva Prep - Page 2

Database Design and Relationships

Main parent tables

- Tenants is the parent for company data.
- Users belongs to one tenant and has a role.
- Subscription_Plans is linked to tenants and billing.
- Pipeline_Stages belongs to a tenant and is used by opportunities.

CRM working tables

- Leads, Contacts, Accounts, Opportunities store sales data.
- Activities track tasks, meetings, and calls.
- Support_Tickets and Ticket_Comments handle support.
- Audit_Logs and Notifications support monitoring.

Relationship	How to explain it
Tenants to Users	One tenant can have many users. Users.tenant_id points to Tenants.tenant_id.
Tenants to Leads/Contacts/Accounts	Each customer record belongs to one company, so tenant_id keeps data separate.
Users to Leads/Contacts/Accounts	assigned_user_id shows which employee is responsible for that record.
Accounts to Contacts	One account/company can have many contact persons.
Opportunities to Account/Contact/Stage	A deal belongs to an account, may have a contact, and is placed in a pipeline stage.
Tickets to Comments	One support ticket can have many comments.
Lead_Conversions	Connects the old lead with the new contact/account/opportunity after conversion.
Billing to Tenants and Plans	Billing_Records tells which tenant paid for which subscription plan.

Constraints answer

The database should use primary keys for every table, foreign keys for relationships, NOT NULL for required fields, UNIQUE for values like username, email, tenant domain, or invoice number, DEFAULT for dates like created_at, and CHECK constraints for valid statuses, roles, amount values, and probability percentages.

Normalization up to BCNF

Q How did you normalize the database?

A I separated each real-world thing into its own table: tenants, users, leads, accounts, contacts, opportunities, plans, billing, tickets, and so on. This avoids repeating the same data again and again.

Q Give a 3NF or BCNF example from your project.

A In Opportunities I store stage_id, not stage_name and probability directly. Stage details are in Pipeline_Stages. So stage_id determines the stage details, and no non-key field depends on another non-key field.

TenantCRM Viva Prep - Page 3

SQL Queries Your Teacher May Ask

Q Where are CRUD operations used?

A CRUD is used in almost every module. For example, Leads has add lead, get leads, update lead, and delete lead. Contacts, accounts, users, tickets, plans, opportunities, and billing also have CRUD routes.

Create and read example

```
INSERT INTO Leads
(tenant_id, first_name, last_name,
email, phone,
company, source, status, notes,
assigned_user_id)
VALUES
(@tenant_id, @first_name, @last_name,
@email,
@phone, @company, @source, @status,
@notes,
@assigned_user_id);

SELECT l.*, u.first_name + ' ' +
u.last_name AS assigned_user
FROM Leads l
LEFT JOIN Users u ON l.assigned_user_id
= u.user_id
WHERE l.tenant_id = @tenant_id;
```

Update and delete example

```
UPDATE Opportunities
SET status = @status,
last_updated = GETDATE()
WHERE opportunity_id = @opportunity_id;

DELETE FROM Support_Tickets
WHERE ticket_id = @ticket_id;
```

Q Which joins are used in your project?

A I use JOIN and LEFT JOIN to show readable names instead of only IDs. For example, opportunities join tenants, accounts, contacts, users, and pipeline stages so the dashboard can show company name, contact name, assigned user, and stage name.

```
SELECT
o.opportunity_name,
o.deal_value,
t.company_name,
a.account_name,
c.first_name + ' ' + c.last_name AS contact_name,
u.first_name + ' ' + u.last_name AS assigned_user,
ps.stage_name
FROM Opportunities o
LEFT JOIN Tenants t ON o.tenant_id = t.tenant_id
LEFT JOIN Accounts a ON o.account_id = a.account_id
LEFT JOIN Contacts c ON o.contact_id = c.contact_id
LEFT JOIN Users u ON o.assigned_user_id = u.user_id
LEFT JOIN Pipeline_Stages ps ON o.stage_id = ps.stage_id;
```

Q Where did you use grouping?

A The dashboard uses GROUP BY for reports. Revenue is grouped by tenant company, and pipeline value is grouped by tenant company. This gives summary values, not only raw records.

```
SELECT t.company_name,  
       SUM(b.amount) AS total_revenue,  
       COUNT(b.billing_id) AS total_invoices  
FROM Billing_Records b  
JOIN Tenants t ON b.tenant_id = t.tenant_id  
WHERE b.payment_status = 'paid'  
GROUP BY t.company_name  
ORDER BY total_revenue DESC;
```

TenantCRM Viva Prep - Page 4

Triggers and Transactions

Important demo note

Your route files show the audit module and the lead conversion workflow. If your SQL Server has manually created triggers or transactions, show them from SQL Server during viva. If the teacher asks for the concept, use the answers below.

Q What trigger is used in your project?

A The trigger concept is used for audit logging. When an important table such as Leads, Users, Opportunities, or Billing_Records changes, a database trigger can automatically insert a record into Audit_Logs with action type, table name, tenant_id, user_id, and time. This gives a history without writing audit code on every page.

Q Why use a trigger instead of normal code?

A A trigger runs automatically inside the database. Even if the change comes from another route or admin tool, the audit record is still created. This protects consistency.

```
CREATE TRIGGER trg_Leads_Audit
ON Leads
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    INSERT INTO Audit_Logs (tenant_id, action_type, table_name, performed_at)
    SELECT tenant_id, 'LEAD_CHANGED', 'Leads', GETDATE()
    FROM inserted;
END;
```

Q Where is transaction needed in your project?

A Lead conversion is the best transaction example. It inserts a row into Lead_Conversions and also updates the original lead status to converted. Both steps must succeed together. If the insert succeeds but the update fails, the database would become inconsistent, so we rollback.

```
BEGIN TRANSACTION;

INSERT INTO Lead_Conversions
(tenant_id, lead_id, contact_id, account_id,
opportunity_id, converted_by, notes)
VALUES
(@tenant_id, @lead_id, @contact_id, @account_id,
@opportunity_id, @converted_by, @notes);

UPDATE Leads
SET status = 'converted',
    last_updated = GETDATE()
WHERE lead_id = @lead_id;

COMMIT;
-- If any step fails: ROLLBACK;
```

Q Give another transaction example.

- A** Registration can also be treated as a transaction because the system creates a tenant first and then creates a tenant admin user. If the user creation fails, the tenant creation should also rollback so no empty company is left.

TenantCRM Viva Prep - Page 5

Likely Viva Questions with Simple Answers

Q What problem does your CRM solve?

A It helps companies manage customers from first lead to sales opportunity, support ticket, billing, and follow-up activities in one system.

Q What does multi-tenant mean?

A Many companies use the same application and database, but every company has its own `tenant_id`. Queries filter by `tenant_id` so one company does not see another company's data.

Q How does login work?

A The login route checks username and password in `Users`, joins `Tenants` to get company name, checks `is_active`, updates `last_login`, and sends the user role back to the frontend.

Q How are roles handled?

A The user role decides the dashboard. `super_admin` sees platform-wide pages, `tenant_admin` sees one tenant's data, and `sales_agent` sees assigned records.

Q How do you prevent SQL injection?

A The backend uses parameterized inputs like `.input('tenant_id', sql.Int, tenant_id)` instead of directly joining user text into SQL. This is safer.

Q Why did you separate Subscription_Plans from Tenants?

A Because plan details like `max_users`, `max_contacts`, `price`, and `features` should be stored once. Tenants only store `subscription_plan_id`. This reduces duplication and supports normalization.

Q Why did you create Pipeline_Stages?

A Pipeline stages are reusable steps like `new`, `qualified`, `proposal`, `won`, or `lost`. Opportunities only store `stage_id`, so stage names and probabilities are managed in one table.

Q What is Lead_Conversions used for?

A It keeps the history of converting a lead into a contact, account, or opportunity. It also records who converted it and when.

Q What happens when a ticket is resolved?

A The ticket status is updated and `resolved_at` is set. That records both the current state and the time of resolution.

Q Where is GETDATE used?

A GETDATE is used for time tracking, such as last_login, last_updated, notification read_at, audit performed_at, and resolved_at.

TenantCRM Viva Prep - Page 6

More Questions They Can Ask

Q How does the dashboard work?

A It calls backend routes that count leads, contacts, tickets, and tenants. It also calculates revenue and pipeline value using SUM, COUNT, AVG, GROUP BY, and JOIN.

Q Why use LEFT JOIN?

A LEFT JOIN still shows the main record even if optional related data is missing, such as a lead without an assigned user.

Q What is the primary key?

A It uniquely identifies each row. Examples are tenant_id, user_id, lead_id, contact_id, account_id, opportunity_id, and ticket_id.

Q What is a foreign key?

A It connects two tables. For example, Leads.tenant_id connects a lead to Tenants, and Opportunities.stage_id connects a deal to Pipeline_Stages.

Q Why have Custom_Fields?

A Every tenant may need extra fields. Instead of changing table columns each time, Custom_Fields stores tenant-specific field definitions.

Q Why have Email_Integrations?

A It stores each user's email provider and address, so the CRM can later support email sync and customer communication tracking.

Q What is audit logging?

A Audit logging means keeping a record of important actions, such as who changed data, what table changed, and when it happened.

Q What improvement would you add later?

A I would add password hashing, JWT/session security, stricter database constraints, full transaction wrapping for multi-step operations, and more reports.

Safe closing answer

The main database idea in my project is that tenant_id separates company data, foreign keys connect related CRM modules, joins create readable screens, grouping creates reports, and transactions protect multi-step work like lead conversion.

Last-minute checklist

Check	What to prepare before demo
Server	Run the Node server and confirm <code>http://localhost:3000/landing.html</code> opens.
Database	Make sure SQL Server instance and MultiTenant_CRM database are running.
Login users	Keep one username/password ready for each role.
Demo data	Keep at least one tenant, lead, contact, account, opportunity, ticket, billing record, and notification ready.
SQL proof	Be ready to show one CRUD query, one join query, one GROUP BY query, one trigger, and one transaction.

TenantCRM Viva Prep - Page 7